INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DE SÃO PAULO

CÂMPUS GUARULHOS

GUSTAVO DA SILVA COELHO

RAISSA CARLA FERREIRA DA SILVA

Sistema de apoio à análise de dados com processamento de linguagem natural (NLP)

GUARULHOS

ANO

GUSTAVO DA SILVA COELHO
RAISSA CARLA FERREIRA DA SILVA

Sistema de apoio à análise de dados com processamento de linguagem natural (NLP)

Relatório Técnico apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, como parte dos requisitos para a obtenção do grau de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Orientador: Prof. Me. Giovani Fonseca Ravagnani Disperati

GUARULHOS

ANO

Catalogação na fonte
Biblioteca IFSP Câmpus Guarulhos
Dados fornecidos pelo(a) autor(a)

_ERROCOD ,
IGOCUTTE R_ _ / _. Guarulhos: [s.n.], 2024.
100 f. il.

Orientador: _
Co-orientador: _

Trabalho de Conclusão de Curso (Tecnologia em Análise e Desenvolvimento de Sistemas) - Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, IFSP, 2024.

Inclui bibliografia.

1. _. 2. _. 3. _. 4. _. 5. _. I. Instituto Federal de Educação, Ciência e Tecnologia de São Paulo II. Título.

CDD 004.21

ATA N.º /2024 - DAE-GRU/DRG/GRU/IFSP

ATA DE DEFESA DE MONOGRAFIA

Na presente data realizou-se a sessão pública de defesa da Monografia intitulada

Membros	IES	Presença (Sim/Não)	Aprovação/Conceito *(Quando Exigido)*

Observações:

A banca examinadora, tendo terminado a apresentação do conteúdo da monografia, passou à arguição do candidato. Em seguida, os examinadores reuniram-se para avaliação e deram o parecer final sobre o trabalho apresentado pelo aluno, tendo sido atribuído o seguinte resultado:

[ ] Aprovado [ ] Reprovado

Proclamados os resultados pelo presidente da banca examinadora, foram encerrados os trabalhos e, para constar, eu lavrei a presente ata que assino juntamente com os demais membros da banca examinadora.

Campus Guarulhos

Documento assinado eletronicamente.

Documento assinado eletronicamente por:

-
-
-
-

Este documento foi emitido pelo SUAP em 05/07/2024. Para comprovar sua autenticidade, faça a leitura do QRCode ao lado ou acesse https://suap.ifsp.edu.br/autenticar-documento/ e forneça os dados abaixo:

Código Verificador:
Código de Autenticação:

Dedicatória (Opcional). Não digite a palavra Dedicatória. Texto no qual o autor do trabalho oferece homenagem ou dedica o seu trabalho a alguém (não usar ponto final)

AGRADECIMENTOS

Agradecimento (opcional). Folha que contém manifestação de reconhecimento a pessoas e/ou instituições que realmente contribuíram com o autor.

Epígrafe (Opcional). Pensamentos retirados de um livro, uma música, um poema, normalmente relacionado ao tema do trabalho, seguida de indicação de autoria. As epígrafes podem ser colocadas também nas folhas de abertura de cada capítulo.

"*Here's to the crazy ones. The misfits. The rebels. The troublemakers. The round pegs in the square holes. The ones who see things differently. They're not fond of rules. And they have no respect for the status quo. You can quote them, disagree with them, glorify or vilify them. About the only thing you can't do is ignore them. Because they change things. They push the human race forward. And while some may see them as the crazy ones, we see genius. Because the people who are crazy enough to think they can change the world, are the ones who do.*".

RESUMO

Elemento obrigatório, constituído de uma sequência de frases concisas e objetivas, fornecendo uma visão rápida e clara do conteúdo do estudo. O texto deverá conter entre 150 a 250 palavras e ser antecedido pela referência do estudo. Também, não deve conter citações e deverá ressaltar o objetivo, o método, os resultados e as conclusões. O resumo deve ser redigido em parágrafo único, seguido das palavras representativas do conteúdo do estudo, isto é, palavras-chave, em número de três a cinco, separadas entre si por ponto e finalizadas também por ponto. Usar o verbo na terceira pessoa do singular, com linguagem impessoal (pronome SE), bem como fazer uso, preferencialmente, da voz ativa.

Palavras-chave: palavra-chave 1; palavra-chave 2; palavra-chave 3; palavra-chave n.

LISTA DE ILUSTRAÇÕES

Figura 1 – Cronograma previsto de outubro de 2026 a junho de 2027 54
Figura 2 – Diagrama de casos de uso do MVP (V1) 65
Figura 3 – Diagrama de classes – modelo de domínio 72
Figura 4 – Diagrama de classes – módulos do backend 74

LISTA DE QUADROS

Quadro 1 – Atores do sistema 63
Quadro 2 – Casos de uso e requisitos atendidos 65
Quadro 3 – Rastreabilidade dos requisitos funcionais 68
Quadro 4 – Critérios de verificação dos requisitos não funcionais 69
Quadro 5 – Classes do modelo de domínio 73
Quadro 6 – Classes de serviço do backend 75

LISTA DE TABELAS

LISTA DE ABREVIATURAS E SIGLAS

1D	Uma dimensão
2D	Duas dimensões
3D	Três dimensões

LISTA DE SÍMBOLOS

α	Letra grega minúscula Alfa
β	Letra grega minúscula Beta

SUMÁRIO

1	INTRODUÇÃO	29
1.1	Justificativa	30
1.2	Objetivos	31
1.2.1	Objetivo Geral	31
1.2.2	Objetivos Específicos	31
2	MÉTODO DE DESENVOLVIMENTO	33
2.1	Visão Geral do Projeto	33
2.2	Requisitos	35
2.2.1	Requisitos Funcionais	35
2.2.2	Requisitos Não Funcionais	36
2.2.3	Não Escopo	38
2.3	Processo de Desenvolvimento de Software	38
2.4	Tecnologias	41
2.4.1	Python	41
2.4.2	FastAPI	41
2.4.3	Pydantic	42
2.4.4	PostgreSQL	42
2.4.5	SQLAlchemy	42
2.4.6	React, Vite e TypeScript	42
2.4.7	Electron	43
2.4.8	Modelos de Linguagem	44
2.4.9	Git e GitHub	44
2.4.10	Execução Local	44
3	RESULTADOS E DISCUSSÃO	47
4	CONCLUSÕES PARCIAIS	49
4.1	Papel do Contexto na Solução	49
4.2	Definições Consolidadas e Pendências	50
4.3	Avaliação Prevista e Evoluções	50
5	CRONOGRAMA E GESTÃO DE PESSOAS	53
5.1	Gestão de Pessoas	53
5.2	Cronograma das Atividades	54
5.3	Planejamento da Avaliação	55

REFERÊNCIAS 59

APÊNDICES 61

APÊNDICE A – MODELO DE REQUISITOS DO SISTEMA 63
A.1 Atores 63
A.2 Regras de Negócio 63
A.3 Casos de Uso 64
A.3.1 UC01 – Configurar conexão com o banco 64
A.3.2 UC02 – Identificar estrutura do banco 65
A.3.3 UC03 – Gerenciar consentimento de amostragem 66
A.3.4 UC04 – Realizar consulta em linguagem natural 66
A.3.5 UC05 – Validar solicitação e consulta 67
A.3.6 UC06 – Executar consulta e apresentar resultado 67
A.3.7 UC07 – Corrigir consulta automaticamente 68
A.3.8 UC08 – Tratar falha 68
A.4 Rastreabilidade dos Requisitos 68
A.5 Critérios de Verificação dos Requisitos Não Funcionais 68

APÊNDICE B – MODELO ESTRUTURAL DO SISTEMA 71
B.1 Modelo de Domínio 71
B.2 Modelo de Módulos do Backend 71
B.3 Considerações sobre o Modelo 71

APÊNDICE C – MODELO COMPORTAMENTAL DO SISTEMA . . 77

APÊNDICE D – MODELO DA ARQUITETURA DO SISTEMA . . . 79

APÊNDICE E – MODELO DE DADOS DO SISTEMA 81

APÊNDICE F – OUTRO APÊNDICE 83

ANEXOS 85

ANEXO A – TÍTULO DO ANEXO A 87

ANEXO B – TÍTULO DO ANEXO B 89

ANEXO C – TÍTULO DO ANEXO C 91

ANEXO D – TÍTULO DO ANEXO D 93

ANEXO E – TÍTULO DO ANEXO E 95

1 INTRODUÇÃO

O acesso rápido a informações confiáveis é importante para o acompanhamento de processos e para a tomada de decisões nas organizações. Entretanto, os dados corporativos costumam estar armazenados em bancos de dados relacionais cuja consulta exige conhecimento de linguagens específicas, como a Structured Query Language (SQL). Em consequência, colaboradores que conhecem o contexto do negócio, mas não dominam SQL, frequentemente dependem de profissionais de Tecnologia da Informação (TI) para obter relatórios e responder a perguntas operacionais. Essa dependência pode aumentar o tempo necessário para acessar informações e direcionar a equipe técnica a solicitações repetitivas de extração de dados.

Os avanços em Inteligência Artificial (IA), especialmente no Processamento de Linguagem Natural (PLN), ampliaram as possibilidades de interação entre pessoas e sistemas computacionais. Nesse contexto, a área denominada *text-to-SQL* investiga métodos capazes de interpretar uma pergunta formulada em linguagem natural e transformá-la em uma consulta SQL compatível com a estrutura de um banco de dados (Deng; Chen; Zhang, 2022). Essa abordagem pode tornar a consulta de dados mais acessível, mas ainda envolve desafios relacionados à compreensão da intenção do usuário, à associação entre os termos da pergunta e o esquema do banco, à correção da consulta produzida e à generalização para diferentes estruturas de dados (Yu et al., 2018).

Este trabalho tem como tema a aplicação de IA na consulta a bancos de dados e delimita-se ao desenvolvimento de uma plataforma web para bancos de dados relacionais. Por meio da plataforma, o usuário poderá formular, em português e sem escrever SQL, perguntas como “Quais foram os cinco produtos mais vendidos em julho?”. A aplicação enviará a pergunta e as informações necessárias sobre o esquema do banco a um modelo de IA, receberá uma consulta SQL, verificará sua sintaxe e sua segurança, executará apenas consultas de leitura autorizadas e apresentará o resultado de forma compreensível.

O problema que orienta o projeto pode ser expresso pela seguinte questão: como permitir que usuários sem conhecimento técnico em SQL consultem dados relacionais por meio de linguagem natural, preservando a segurança, a integridade e o controle de acesso às informações? Para responder a essa questão, a solução proposta combinará geração de SQL por IA com uma camada de validação independente da resposta do modelo. Serão aceitas exclusivamente consultas do tipo `SELECT`, e comandos capazes de alterar estruturas ou registros, como `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER` e `TRUNCATE`, deverão ser bloqueados antes da execução.

O protótipo contemplará autenticação, perfis de acesso, cadastro controlado de conexões, histórico de consultas e apresentação dos resultados em tabelas e, quando pertinente, em gráficos. Também poderá explicar em linguagem natural a consulta gerada, sugerir perguntas com base no esquema disponível, permitir o reaproveitamento de consultas frequentes e exportar resultados. A interface será desenvolvida com tecnologias web, enquanto o *back-end* será implementado majoritariamente em Python, com FastAPI ou Flask, a ser definido após avaliação técnica. O sistema será avaliado em cenários controlados e com bancos de dados de teste, não se propondo, nesta etapa, a substituir ferramentas completas de inteligência de negócios nem a permitir alterações nos dados consultados.

1.1 Justificativa

Sob a perspectiva teórica, o projeto se justifica pela relevância do problema *text-to-SQL* para as áreas de PLN e bancos de dados. A tradução de uma pergunta para SQL não corresponde apenas à substituição de palavras: o sistema precisa identificar a intenção expressa, relacionar entidades e atributos ao esquema disponível e construir uma consulta sintática e semanticamente adequada. A literatura apresenta avanços importantes nessa área, mas também reconhece desafios de interpretação, geração e avaliação das consultas (Deng; Chen; Zhang, 2022). O conjunto de dados Spider, por exemplo, foi elaborado para avaliar consultas complexas e a generalização entre diferentes domínios e esquemas de bancos de dados, evidenciando a necessidade de soluções que sejam testadas além de exemplos simples ou de uma única base (Yu et al., 2018).

Em termos acadêmicos, o desenvolvimento da plataforma possibilita integrar conhecimentos de engenharia de software, bancos de dados, segurança da informação, desenvolvimento web e IA em um problema aplicado. Além de construir um protótipo funcional, o trabalho permitirá observar em quais tipos de pergunta a geração automática produz resultados corretos, quais erros ocorrem com maior frequência e em que medida as verificações implementadas impedem consultas inadequadas. Assim, a avaliação do sistema poderá produzir evidências sobre as possibilidades e as limitações do uso de modelos de IA como interface para dados relacionais.

Sob a perspectiva prática e de mercado, a solução busca aproximar os dados dos profissionais que precisam utilizá-los, mas que não possuem formação técnica em SQL. Ao permitir consultas em linguagem cotidiana, a plataforma poderá reduzir o tempo gasto em solicitações simples de relatórios, dar maior autonomia aos usuários e liberar a equipe de TI para atividades de maior complexidade. A apresentação em tabelas, gráficos e explicações em português também poderá facilitar a interpretação dos resultados e ampliar o uso das informações disponíveis no processo decisório.

Por fim, a adoção de IA nesse contexto exige cuidados específicos, pois uma consulta incorreta pode apresentar resultados enganosos, consumir recursos excessivos ou afetar informações sensíveis. Por esse motivo, a proposta não tratará a resposta do modelo como uma instrução automaticamente confiável. A restrição a operações de leitura, a validação prévia do SQL, o controle de permissões, o registro do histórico e o uso de bancos de teste constituem partes centrais da solução. Dessa maneira, o projeto procura conciliar facilidade de uso e inovação com mecanismos objetivos de segurança e rastreabilidade.

1.2 Objetivos

1.2.1 Objetivo Geral

Desenvolver e avaliar um protótipo de plataforma web que permita a usuários sem domínio de SQL consultar bancos de dados relacionais por meio de perguntas em linguagem natural, utilizando IA para gerar consultas de leitura, validando-as antes da execução e apresentando os resultados de maneira segura e compreensível.

1.2.2 Objetivos Específicos

- Revisar conceitos, técnicas e trabalhos relacionados a PLN, modelos de IA e conversão de linguagem natural para SQL;
- Definir a arquitetura da plataforma e o fluxo de comunicação entre interface web, *back-end*, modelo de IA e banco de dados;
- Desenvolver uma interface web intuitiva para o envio de perguntas e a visualização das respostas, considerando usuários sem conhecimento técnico em SQL;
- Implementar o *back-end* majoritariamente em Python, adotando FastAPI ou Flask conforme os critérios técnicos definidos durante o projeto;
- Integrar um modelo de IA capaz de interpretar perguntas em português e gerar consultas SQL compatíveis com o esquema do banco conectado;
- Implementar uma camada independente de validação sintática e de segurança que autorize somente consultas de leitura e bloqueie comandos de alteração de dados ou de estrutura;
- Implementar autenticação, perfis de acesso, controle de permissões e registro do histórico das consultas realizadas;
- Apresentar os resultados em tabelas e, quando adequado, em gráficos, acompanhados de uma explicação em linguagem natural sobre a consulta executada;

- Disponibilizar recursos complementares previstos no escopo, como sugestões de perguntas, consultas favoritas, exportação de resultados e tentativa controlada de correção de consultas com erro; e
- Avaliar o protótipo em bancos de dados de teste, considerando a correção das consultas geradas, a efetividade das regras de segurança, o tempo de execução e a compreensão dos resultados apresentados.

2 MÉTODO DE DESENVOLVIMENTO

Este capítulo apresenta o método de desenvolvimento adotado para a construção do sistema proposto, contemplando a visão geral da solução, seus requisitos funcionais e não funcionais, o processo de desenvolvimento de software e as principais tecnologias previstas.

O projeto será conduzido de maneira incremental, priorizando inicialmente a validação do fluxo central da solução: identificação automática da estrutura de um banco de dados PostgreSQL, interpretação de uma pergunta formulada em linguagem natural, geração de uma consulta SQL segura, execução da consulta e apresentação do resultado ao usuário. A partir dessa base, funcionalidades adicionais poderão ser incorporadas progressivamente.

A solução será desenvolvida por uma equipe composta por dois integrantes, utilizando um processo baseado em Kanban e adaptado às características e à escala do projeto acadêmico. O desenvolvimento será acompanhado por meio de um backlog, histórias de usuário, reuniões periódicas e validações progressivas dos incrementos produzidos.

2.1 Visão Geral do Projeto

O projeto propõe o desenvolvimento de uma aplicação capaz de permitir consultas a bancos de dados PostgreSQL por meio de linguagem natural. O usuário deverá fornecer as informações necessárias para conexão com um banco de dados e, posteriormente, poderá formular perguntas textuais relacionadas aos dados armazenados. A aplicação será responsável por interpretar a solicitação, gerar uma consulta SQL compatível com a estrutura identificada, executá-la de maneira controlada e apresentar o resultado obtido.

Um dos principais componentes da solução será o módulo de descoberta e introspecção do banco de dados, denominado internamente como *crawler*. Esse módulo realizará automaticamente o levantamento da estrutura acessível por meio da conexão fornecida pelo usuário, percorrendo os esquemas disponíveis e identificando, no mínimo, tabelas, colunas, tipos de dados, chaves primárias e chaves estrangeiras.

A descoberta estrutural será realizada de maneira determinística, sem depender diretamente de um modelo de linguagem para identificar os elementos básicos do banco de dados. As informações encontradas serão organizadas em uma representação estruturada e posteriormente utilizadas como contexto pelas etapas responsáveis pela

interpretação da solicitação e pela geração da consulta SQL.

Além dos metadados estruturais, o sistema poderá utilizar pequenas amostras dos registros armazenados para auxiliar na compreensão do significado prático das tabelas e colunas. Como essas amostras podem conter informações sensíveis, sua utilização dependerá de consentimento explícito do usuário.

Quando autorizada, a amostragem será limitada a, no máximo, cem registros por operação de amostragem por padrão, podendo ser incrementada se de acordo com o usuário. Caso o usuário não autorize o acesso aos registros, o sistema continuará funcionando exclusivamente a partir das informações estruturais obtidas durante a introspecção do banco.

De maneira geral, a arquitetura conceitual da solução será composta pelo seguinte fluxo:

Aplicação $\rightarrow$ Interface React $\rightarrow$ API/Orquestração $\rightarrow$ Descoberta e Introspecção do Banco $\rightarrow$ Preparação do Contexto $\rightarrow$ Interpretação da Solicitação $\rightarrow$ Geração Text-to-SQL $\rightarrow$ Guardrails e Validação $\rightarrow$ Execução Read-only $\rightarrow$ Processamento do Resultado $\rightarrow$ Apresentação ao Usuário.

A aplicação web será disponibilizada como software desktop, permitindo que a interface construída com tecnologias web seja executada tanto online em nuvem, quanto diretamente no computador do usuário. Essa característica será especialmente relevante em cenários nos quais o banco PostgreSQL esteja hospedado localmente ou disponível apenas na rede interna do usuário.

A etapa de interpretação da solicitação também será utilizada para identificar pedidos incompatíveis com o propósito da aplicação. Solicitações que envolvam inserção, atualização, exclusão ou alteração estrutural do banco não deverão prosseguir para execução.

O desenvolvimento da solução será organizado em quatro incrementos principais.

A **V0** corresponderá a uma prova de conceito, cujo objetivo será validar tecnicamente o fluxo essencial da aplicação. Nessa etapa serão avaliados principalmente a conexão com PostgreSQL, a introspecção da estrutura do banco, o recebimento de uma pergunta em linguagem natural, a geração da consulta SQL, a validação da operação e a apresentação do resultado obtido.

A **V1** corresponderá ao Produto Mínimo Viável (MVP) e constituirá a entrega-alvo do trabalho. Nessa versão, o fluxo validado na V0 será integrado a uma interface

gráfica, permitindo que o usuário configure uma conexão PostgreSQL, autorize ou não a utilização de amostras, realize perguntas e visualize os resultados em formato tabular.

A **V2** será considerada uma evolução prevista da solução e acrescentará uma etapa de interpretação dos resultados obtidos, permitindo que o sistema apresente também uma explicação textual dos dados retornados.

Por fim, a **V3** representará uma evolução voltada à visualização de dados. Nessa etapa, o sistema poderá sugerir formas adequadas de representação gráfica dos resultados, enquanto a interface será responsável pela renderização dessas visualizações.

As versões V2 e V3 serão tratadas como possibilidades de evolução e não como requisitos obrigatórios para a conclusão do MVP.

2.2 Requisitos

Os requisitos de software representam os serviços que o sistema deverá oferecer, bem como as restrições que deverão ser consideradas em seu desenvolvimento. Segundo (Sommerville, 2003; Sommerville, 2007), os requisitos podem ser classificados em funcionais, relacionados diretamente às funcionalidades e comportamentos esperados do sistema, e não funcionais, associados a restrições e propriedades de qualidade da solução.

Além desses requisitos, também será explicitado o não escopo do projeto, delimitando funcionalidades que não serão consideradas parte da entrega prevista. A definição dessas fronteiras contribui para o controle da complexidade e para a manutenção dos objetivos estabelecidos para o projeto (PRESSMAN, 2016).

2.2.1 Requisitos Funcionais

Os requisitos funcionais inicialmente definidos para a aplicação são apresentados a seguir:

- **RF01 – Conexão com banco de dados:** o sistema deverá permitir que o usuário informe os dados necessários para conexão com um banco de dados PostgreSQL.
- **RF02 – Identificação de esquemas:** o sistema deverá identificar automaticamente os esquemas acessíveis utilizando a conexão fornecida.
- **RF03 – Introspecção estrutural:** o sistema deverá identificar automaticamente tabelas, colunas, tipos de dados, chaves primárias e chaves estrangeiras existentes nas estruturas acessíveis.

- **RF04 – Consentimento para amostragem:** o sistema deverá solicitar autorização explícita do usuário antes de utilizar registros do banco como contexto enviado ao modelo de linguagem.
- **RF05 – Limitação das amostras:** quando a utilização de registros for autorizada, o sistema deverá limitar cada amostragem a, no máximo, dez registros.
- **RF06 – Entrada em linguagem natural:** o sistema deverá permitir que o usuário formule perguntas sobre os dados utilizando linguagem natural.
- **RF07 – Validação da intenção:** o sistema deverá identificar solicitações incompatíveis com o escopo exclusivamente de leitura da aplicação.
- **RF08 – Geração de consulta SQL:** o sistema deverá gerar consultas SQL de leitura a partir da pergunta realizada pelo usuário e das informações disponíveis sobre a estrutura do banco de dados.
- **RF09 – Execução da consulta:** o sistema deverá executar as consultas consideradas permitidas sobre o banco PostgreSQL informado.
- **RF10 – Apresentação do resultado:** o sistema deverá apresentar o resultado obtido em formato tabular por meio da interface gráfica da aplicação.
- **RF11 – Correção automática de consultas:** quando uma consulta considerada permitida apresentar falha durante sua geração ou execução, o sistema poderá realizar até três tentativas automáticas de correção.
- **RF12 – Tratamento de falha:** caso a consulta não possa ser concluída após o limite de tentativas estabelecido, o sistema deverá encerrar a operação de maneira controlada e informar a falha ao usuário.

Como requisitos evolutivos, posteriores ao MVP, estarão previstos a geração de explicações textuais sobre os resultados na V2 e a geração ou sugestão de visualizações gráficas na V3.

2.2.2 Requisitos Não Funcionais

Os principais requisitos não funcionais definidos são:

- **RNF01 – Segurança:** o sistema deverá operar exclusivamente com consultas de leitura, impedindo operações que realizem escrita ou alteração estrutural no banco de dados.

- **RNF02 – Privacidade:** registros armazenados no banco somente poderão ser enviados como contexto a um provedor de inteligência artificial mediante consentimento explícito do usuário.

- **RNF03 – Independência da amostragem:** a recusa do usuário em disponibilizar amostras de registros não poderá impedir o funcionamento básico da aplicação, que deverá permanecer capaz de operar utilizando apenas os metadados estruturais do banco.

- **RNF04 – Modularidade:** a arquitetura deverá manter separadas as responsabilidades relacionadas à introspecção do banco, preparação de contexto, geração de SQL, validação, execução e apresentação dos resultados.

- **RNF05 – Baixo acoplamento ao modelo de linguagem:** a integração com inteligência artificial deverá ser projetada de maneira que permita a substituição do modelo ou do provedor sem exigir a reestruturação completa da aplicação.

- **RNF06 – Tratamento de erros:** falhas relacionadas à conexão com o banco, comunicação com o provedor de inteligência artificial, geração da consulta ou execução do SQL deverão ser tratadas de forma controlada.

- **RNF07 – Observabilidade:** o sistema deverá registrar informações suficientes para permitir a análise de seu comportamento durante os testes, incluindo, quando aplicável, latência, erros, quantidade de tentativas e execução das consultas.

- **RNF08 – Usabilidade:** a V1 deverá disponibilizar uma interface gráfica executável no computador do usuário, permitindo realizar o fluxo principal da aplicação sem necessidade de interação direta com código-fonte ou terminal.

Durante a validação da aplicação também serão consideradas métricas relacionadas ao comportamento operacional do sistema. Entre elas estarão a latência entre a solicitação e a resposta, a ocorrência de erros, o volume de consultas processadas e possíveis limitações de recursos.

Devido ao caráter local e à baixa concorrência esperada para o MVP, as métricas de latência e erros possuirão maior relevância inicial. Também serão avaliadas métricas específicas relacionadas à qualidade da solução, tais como taxa de respostas corretas, taxa de execução SQL bem-sucedida, taxa de bloqueio correto de operações proibidas, quantidade de consultas que necessitaram de novas tentativas e número médio de tentativas realizadas.

2.2.3 Não Escopo

Com o objetivo de manter o desenvolvimento compatível com o tempo, tamanho da equipe e objetivos acadêmicos do projeto, algumas funcionalidades serão explicitamente excluídas do escopo obrigatório.

Não fazem parte do escopo do MVP:

- inserção de novos registros no banco de dados;
- atualização de registros existentes;
- exclusão de registros;
- criação, alteração ou remoção de tabelas, esquemas ou demais objetos estruturais;
- suporte obrigatório a sistemas gerenciadores de bancos de dados diferentes do PostgreSQL;
- suporte inicial a bancos de dados NoSQL;
- criação de dashboards completos;
- treinamento de um modelo de linguagem próprio;
- implantação obrigatória da aplicação em ambiente de nuvem ou disponibilização como aplicação web pública;
- construção obrigatória de um catálogo semântico de negócio;
- utilização obrigatória de glossários de negócio;
- utilização de mecanismos baseados em embeddings ou RAG como requisito da V1.

Catálogos semânticos, glossários e mecanismos baseados em embeddings poderão ser avaliados em evoluções posteriores caso se mostrem necessários para melhorar a compreensão de bancos de dados com estruturas ou domínios mais complexos.

2.3 Processo de Desenvolvimento de Software

O desenvolvimento da aplicação será realizado utilizando uma abordagem incremental e iterativa, apoiada por um processo baseado em Kanban e adaptado ao tamanho reduzido da equipe e às características acadêmicas do projeto.

A equipe será composta por dois integrantes. Dessa forma, optou-se por um processo com baixa sobrecarga de gestão e documentação, privilegiando visibilidade das atividades, priorização contínua e execução progressiva das funcionalidades.

O trabalho será organizado a partir de um backlog contendo histórias de usuário, atividades técnicas, correções e demais tarefas necessárias para o desenvolvimento. Os itens serão priorizados conforme sua relevância para o funcionamento da solução e serão selecionados de acordo com a capacidade disponível da equipe.

Serão realizadas reuniões semanais para acompanhamento das atividades, identificação de impedimentos, revisão das prioridades e definição dos próximos itens do backlog. O Kanban será utilizado de maneira adaptada, sem a necessidade de introduzir papéis ou cerimônias que não agreguem valor ao contexto de uma equipe composta por apenas dois integrantes.

O desenvolvimento seguirá os incrementos V0, V1, V2 e V3 definidos anteriormente. A V0 será utilizada principalmente para reduzir os riscos técnicos relacionados à introspecção do PostgreSQL, preparação de contexto, geração de consultas SQL e execução segura. Após a validação desse fluxo, o projeto avançará para a V1, correspondente ao MVP e à principal entrega prevista no TCC, incluindo a integração do backend com uma interface gráfica.

Essa organização também se relaciona com os princípios de customização de processos, ou *tailoring*, estudados durante a disciplina. A implementação gradual de um processo ou solução permite que áreas prioritárias sejam tratadas primeiro, evitando a tentativa de implantação de todos os elementos simultaneamente. No contexto deste projeto, essa ideia será aplicada por meio da divisão da solução em incrementos progressivos, nos quais cada versão adicionará novas capacidades a uma base previamente validada.

Outro princípio considerado será a seleção de artefatos de acordo com o valor que fornecem ao desenvolvimento. Considerando o tamanho da equipe, serão priorizados os documentos e artefatos necessários para apoiar a comunicação, rastreabilidade, implementação e avaliação da solução, evitando a produção de documentação cujo custo de elaboração e manutenção não seja proporcional ao benefício obtido.

Entre os principais artefatos previstos encontram-se:

- backlog do produto;
- histórias de usuário;
- especificação de requisitos;
- documentação da arquitetura;

- diagramas considerados necessários para representar a solução;
- código-fonte versionado;
- casos e resultados de testes;
- relatório técnico do projeto.

A divisão de responsabilidades também será adaptada às características da equipe. Ambos os integrantes participarão do desenvolvimento, especialmente das atividades relacionadas ao backend e ao fluxo central da aplicação. Entretanto, um dos integrantes terá maior concentração nas atividades de organização, documentação e apresentação do projeto, enquanto o outro terá maior concentração na implementação.

Essa divisão não será rígida, e as principais decisões técnicas e atividades de desenvolvimento permanecerão compartilhadas entre os integrantes. Dessa forma, a distribuição de responsabilidades poderá ser representada por uma matriz simplificada, evitando a criação de uma estrutura organizacional desnecessariamente complexa.

O código-fonte será versionado utilizando Git e armazenado em um repositório no GitHub. O versionamento permitirá o acompanhamento das alterações realizadas, a integração do trabalho dos dois integrantes e a recuperação de versões anteriores quando necessário.

A validação do sistema ocorrerá progressivamente durante o desenvolvimento. Será criado um conjunto controlado de perguntas cujos resultados esperados sejam previamente conhecidos, permitindo avaliar o comportamento do sistema diante de diferentes níveis de complexidade.

A avaliação priorizará a correção do resultado retornado, e não necessariamente a igualdade textual entre o SQL produzido pelo sistema e uma consulta de referência. Essa decisão considera que consultas SQL sintaticamente diferentes podem produzir resultados semanticamente equivalentes.

Entre as principais métricas previstas para a avaliação estão:

- taxa de respostas corretas;
- taxa de consultas SQL executadas com sucesso;
- taxa de bloqueio correto de operações proibidas;
- quantidade de consultas que exigiram uma ou mais tentativas de correção;
- número médio de tentativas realizadas;
- latência entre a solicitação do usuário e a apresentação do resultado;

- quantidade de tokens utilizados, quando essa informação for disponibilizada pelo provedor;
- custo estimado por consulta, quando aplicável.

A comparação entre diferentes modelos de linguagem poderá ser realizada internamente durante o desenvolvimento para apoiar a escolha da alternativa que apresente melhor equilíbrio entre qualidade, custo e latência. Entretanto, essa comparação não será considerada, inicialmente, um experimento formal do TCC.

2.4 Tecnologias

A seleção das tecnologias previstas para o desenvolvimento foi orientada pela compatibilidade com o problema proposto, maturidade das ferramentas, facilidade de integração, disponibilidade de bibliotecas e conhecimento prévio da equipe.

A solução será estruturada em duas camadas principais de execução: uma aplicação responsável pela interface com o usuário, construída com React, Vite, TypeScript e Electron, e um backend desenvolvido em Python e FastAPI, responsável pela lógica de negócio, introspecção do banco de dados, integração com modelos de linguagem, validação e execução das consultas.

2.4.1 Python

Python será utilizado como principal linguagem de programação do backend. A linguagem possui um amplo ecossistema de bibliotecas voltadas à integração com bancos de dados, construção de APIs, manipulação de dados e comunicação com modelos de linguagem, características diretamente relacionadas às necessidades da aplicação.

Além disso, sua utilização permitirá concentrar no mesmo ambiente as etapas de introspecção do banco, preparação de contexto, integração com provedores de inteligência artificial e execução das consultas.

2.4.2 FastAPI

O FastAPI será utilizado para a construção da camada de API e para a orquestração dos diferentes módulos do sistema.

O framework será responsável por disponibilizar os serviços utilizados pela interface, receber as solicitações dos usuários e coordenar etapas como conexão com o banco, introspecção, processamento da pergunta, comunicação com o modelo de linguagem e execução das consultas.

2.4.3 Pydantic

O Pydantic será utilizado para definição e validação das estruturas de dados utilizadas pela aplicação.

Sua utilização permitirá estabelecer contratos explícitos para entradas e saídas da API, contribuindo para a validação dos dados recebidos e para a organização da comunicação entre os diferentes componentes do sistema.

2.4.4 PostgreSQL

O PostgreSQL será o Sistema Gerenciador de Banco de Dados oficialmente suportado no escopo inicial do projeto.

A escolha de um único SGBD para o MVP permitirá concentrar os esforços no desenvolvimento do módulo de descoberta, na compreensão de sua estrutura e na geração correta de consultas compatíveis com seu dialeto SQL.

A aplicação não ficará limitada a um banco previamente configurado. O usuário poderá informar os dados de conexão de diferentes bancos PostgreSQL, desde que as credenciais utilizadas possuam acesso às estruturas que se deseja consultar.

2.4.5 SQLAlchemy

O SQLAlchemy será utilizado como camada de acesso ao PostgreSQL e terá papel relevante no módulo de descoberta e introspecção do banco de dados.

Por meio de seus recursos de acesso a metadados e reflexão, será possível obter informações sobre esquemas, tabelas, colunas, tipos e relacionamentos existentes no banco, reduzindo a necessidade de utilizar um modelo de linguagem para tarefas que podem ser realizadas de maneira determinística.

O driver específico utilizado na comunicação entre Python e PostgreSQL será definido durante a implementação, uma vez que sua escolha possui menor impacto sobre a arquitetura geral proposta.

2.4.6 React, Vite e TypeScript

A interface da aplicação será desenvolvida utilizando React em conjunto com TypeScript, tendo o Vite como ferramenta de desenvolvimento e *build* do frontend.

O React será utilizado para estruturar a interface e os componentes responsáveis pela interação do usuário com o sistema. A escolha busca facilitar a construção de uma interface modular, permitindo separar elementos relacionados à configuração da conexão, envio de perguntas, apresentação de erros e visualização dos resultados.

O TypeScript será adotado como linguagem principal do frontend, adicionando tipagem estática ao desenvolvimento e contribuindo para a definição de contratos mais claros entre os componentes da interface e os dados recebidos do backend.

O Vite será utilizado como ferramenta de desenvolvimento, empacotamento e geração dos arquivos utilizados pela interface. Sua utilização permitirá manter um fluxo de desenvolvimento simplificado e integrado ao ecossistema React e TypeScript.

Na V1, a interface terá como objetivo disponibilizar um fluxo simples para que o usuário possa configurar a conexão com o banco PostgreSQL, autorizar ou recusar a utilização de amostras de dados, formular perguntas em linguagem natural e visualizar os resultados retornados.

Funcionalidades mais avançadas relacionadas à interpretação textual e visualização gráfica dos resultados poderão ser adicionadas respectivamente nas versões V2 e V3.

2.4.7 Electron

O Electron será utilizado para empacotar a interface desenvolvida com React, Vite e TypeScript como uma aplicação desktop executável no computador do usuário.

A adoção dessa tecnologia está relacionada principalmente à necessidade de permitir que a aplicação seja executada próxima ao ambiente onde o banco de dados está disponível. Em diferentes cenários, o PostgreSQL que será consultado poderá estar instalado localmente no computador do usuário ou disponível apenas por meio de uma rede interna, o que tornaria menos adequada a dependência de uma aplicação exclusivamente hospedada em ambiente externo.

Por meio do Electron, será possível reutilizar tecnologias tradicionalmente empregadas no desenvolvimento web para construir a interface gráfica da aplicação, mantendo ao mesmo tempo uma experiência de software desktop.

A aplicação desktop atuará como camada de interação com o usuário, enquanto o processamento relacionado à introspecção do banco, integração com modelos de linguagem, geração de SQL, validação e execução das consultas permanecerá concentrado no backend desenvolvido em Python e FastAPI.

Essa separação permitirá manter responsabilidades distintas entre interface e processamento, ao mesmo tempo em que possibilitará a distribuição da solução como uma aplicação executável localmente.

2.4.8 Modelos de Linguagem

Os modelos de linguagem serão responsáveis pelas etapas que exigem interpretação semântica, principalmente a compreensão da solicitação do usuário e a geração das consultas SQL.

A integração será realizada por meio do SDK oficial do provedor selecionado.

Inicialmente, OpenAI e DeepSeek serão considerados como possíveis provedores. A decisão final será realizada durante o desenvolvimento a partir de experimentos internos que considerem fatores como qualidade das consultas produzidas, capacidade de seguir instruções, latência e custo de utilização.

A solução será projetada de maneira a evitar acoplamento excessivo a um único modelo ou provedor, permitindo que essa escolha seja alterada sem exigir modificações significativas nas demais camadas da aplicação.

A descoberta da estrutura do banco permanecerá separada desse processo. Informações que podem ser obtidas deterministicamente, como nomes de tabelas, colunas, tipos e relacionamentos, serão extraídas pelo módulo de introspecção e fornecidas ao modelo como contexto, em vez de delegar ao modelo a responsabilidade de descobrir essas informações.

2.4.9 Git e GitHub

Git será utilizado como sistema de controle de versão do código-fonte, enquanto o GitHub será utilizado para hospedagem do repositório e colaboração entre os integrantes.

Essas ferramentas permitirão registrar o histórico de alterações, integrar o trabalho realizado pela equipe e manter rastreabilidade sobre a evolução da implementação.

2.4.10 Execução Local

A execução local será adotada como estratégia principal para o MVP.

Essa decisão está relacionada à característica do sistema de interagir com bancos PostgreSQL que poderão estar instalados no próprio computador do usuário ou disponíveis apenas em redes internas. Dessa forma, a execução da aplicação no ambiente do usuário reduz a necessidade de expor o banco de dados a serviços externos apenas para possibilitar a conexão.

O Electron será responsável pela disponibilização da interface como aplicação desktop, enquanto o backend em Python e FastAPI será executado localmente e utilizado pela interface para realizar as operações necessárias.

Não será estabelecido como requisito do projeto que a solução esteja hospedada em ambiente de nuvem ou disponível publicamente pela Internet. A implantação em ambientes remotos poderá ser considerada futuramente, mas não será necessária para a validação da proposta apresentada neste trabalho.

3 RESULTADOS E DISCUSSÃO

Texto dos resultados. Deve ser descritas todas as funcionalidades implementadas. É possível adicionar nesta seção reproduções das telas do sistema.

4 CONCLUSÕES PARCIAIS

A etapa de pré-projeto permitiu delimitar a proposta de um sistema de apoio à análise de dados por meio de perguntas em linguagem natural, com PostgreSQL como sistema gerenciador de banco de dados inicialmente suportado. Foram estabelecidos o fluxo principal da aplicação, os requisitos e a organização do desenvolvimento em incrementos. A V0 será utilizada como prova de conceito para verificar a viabilidade técnica da solução, enquanto a V1 constituirá o Produto Mínimo Viável e o compromisso de entrega do trabalho.

No estado atual, o projeto dispõe de documentação e de rascunhos iniciais de módulos, ainda pouco integrados. Dessa forma, as conclusões desta etapa dizem respeito às definições e à organização da proposta. A correção das respostas, a efetividade dos mecanismos de segurança e a adequação da experiência de uso ainda deverão ser verificadas durante o desenvolvimento e a avaliação do protótipo.

4.1 Papel do Contexto na Solução

Uma das principais conclusões do planejamento é que o valor esperado da solução depende fortemente da qualidade do contexto fornecido ao modelo de linguagem. O modelo precisa relacionar os termos utilizados na pergunta com as estruturas e os dados disponíveis no banco, de modo que a geração de SQL corresponda à intenção do usuário.

Por esse motivo, o módulo de descoberta e introspecção do banco, associado à preparação do contexto, constitui um dos componentes centrais da proposta. Sua responsabilidade será identificar esquemas, tabelas, colunas, tipos, chaves e relacionamentos, organizando essas informações para utilização nas etapas de interpretação e geração de consultas. A extração dessas informações estruturais será determinística, sem delegar ao modelo a descoberta da estrutura do banco.

Entretanto, a estrutura do banco não descreve necessariamente todo o significado de negócio dos dados. Nomes, comentários e amostras poderão auxiliar a interpretação, mas códigos internos e conceitos como faturamento, pedido concluído ou cliente ativo podem depender de regras não explicitadas nas tabelas. A suficiência do contexto disponível será, portanto, uma questão a investigar, sem pressupor que a introspecção permita reconstruir automaticamente todas as regras de negócio.

Para a V0 e a V1, a utilização de amostras dependerá de autorização explícita do usuário. Quando autorizada, a coleta será limitada a até cem registros por tabela,

incluindo todas as colunas. Quando recusada, o contexto será composto apenas pelos metadados estruturais disponíveis. Esse consentimento será tratado como uma decisão binária, sem introduzir mecanismos adicionais de seleção de colunas no escopo obrigatório do MVP.

4.2 Definições Consolidadas e Pendências

A implementação utilizará Python e FastAPI no backend e React, Vite e TypeScript na interface. O Electron será empregado para disponibilizar a interface como aplicação desktop, integrada ao backend executado localmente. A execução local será a estratégia principal da V1, sem compromisso de implantação pública em nuvem.

A segurança será apoiada no uso de credenciais com permissões somente de leitura, na validação independente da consulta gerada e no controle da execução pelo backend. A avaliação realizada pelo modelo sobre a solicitação terá caráter complementar e não substituirá os mecanismos de restrição do banco e de validação da aplicação.

O processamento de uma pergunta admitirá, no máximo, três tentativas no total: uma tentativa inicial e até duas correções automáticas. Cada consulta produzida deverá passar novamente pela validação antes da execução. Esgotado esse limite, a aplicação encerrará a operação de forma controlada e informará a falha ao usuário.

Ainda deverão ser definidos o provedor e o modelo de linguagem inicial, os parâmetros operacionais de execução e as metas definitivas de desempenho. A escolha do modelo será apoiada em testes internos, considerando correção, capacidade de seguir instruções, latência e custo, preservando a possibilidade de substituição do provedor.

4.3 Avaliação Prevista e Evoluções

A avaliação utilizará um banco PostgreSQL fictício do domínio comercial, contendo clientes, produtos, pedidos e itens de pedido, com dados e regras de negócio previamente conhecidos pelos autores. As perguntas serão formuladas em português e seus resultados serão comparados com consultas de referência e respostas esperadas.

Será investigado o efeito das amostras na qualidade das respostas por meio da comparação entre duas configurações: contexto formado apenas pelo esquema e contexto formado pelo mesmo esquema acrescido de amostras autorizadas. O modelo, as instruções, as perguntas e os dados do banco serão mantidos constantes entre as configurações. Essa comparação permitirá observar o efeito da inclusão de amostras no cenário avaliado, sem pressupor melhoria obrigatória nem generalização dos resultados

para outros bancos.

As versões V2 e V3 permanecerão como possibilidades de evolução, contemplando, respectivamente, explicações textuais dos resultados e visualizações gráficas. Catálogos semânticos, glossários de negócio e mecanismos de recuperação de contexto poderão ser estudados posteriormente, caso as limitações observadas indiquem sua necessidade. Essas funcionalidades não serão consideradas requisitos obrigatórios para a conclusão da V1.

5 CRONOGRAMA E GESTÃO DE PESSOAS

O planejamento compreende a consolidação do pré-projeto e a elaboração da V0 até novembro de 2026, seguidas pelo desenvolvimento, avaliação e entrega da V1 até junho de 2027. A V0 terá como objetivo verificar a viabilidade técnica do fluxo central, enquanto a V1 constituirá o Produto Mínimo Viável e o compromisso de entrega do trabalho.

Os períodos apresentados são estimativas de planejamento e poderão ser ajustados conforme a disponibilidade da equipe, os resultados dos incrementos e o calendário acadêmico. A data específica da banca técnica da disciplina GPD2 será incorporada quando estiver disponível.

5.1 Gestão de Pessoas

A equipe é composta por Gustavo da Silva Coelho e Raissa Carla Ferreira da Silva. Gustavo concentrará sua atuação no desenvolvimento e na integração técnica, com disponibilidade de até seis horas semanais. Suas responsabilidades principais serão a implementação do backend em Python e FastAPI, a integração com PostgreSQL e com o modelo de linguagem, a construção da interface em React e o empacotamento com Electron, além dos testes técnicos e da correção de falhas.

Raissa concentrará sua atuação na organização do cronograma, acompanhamento das atividades, consolidação da documentação e preparação da apresentação, com disponibilidade estimada entre seis e oito horas semanais. Também apoiará a organização das evidências de avaliação e a comunicação da proposta, de suas contribuições e dos resultados obtidos.

A divisão representa a concentração principal das atividades, sem impedir a colaboração entre os integrantes. Ambos participarão das decisões de escopo e arquitetura, da definição das perguntas de avaliação, da conferência dos resultados esperados, da revisão do relatório e da preparação da defesa.

O projeto contará com a orientação do professor Giovani Fonseca Ravagnani Disperati. Os encontros com o orientador serão acordados conforme sua disponibilidade e os marcos do projeto. A avaliação acadêmica envolverá o orientador e os professores responsáveis pelas etapas do trabalho.

O acompanhamento será realizado por meio de um processo baseado em Kanban, com backlog priorizado e reuniões semanais entre os integrantes. As tarefas deverão indicar responsável, resultado esperado e situação, utilizando estados como a

fazer, em andamento, em revisão e concluída. O código-fonte será versionado com Git e mantido no GitHub. A documentação e os testes acompanharão o desenvolvimento.

5.2 Cronograma das Atividades

A Figura 1 apresenta as atividades previstas até a Avaliação Final do Projeto Integrador. As linhas numeradas correspondem às descrições apresentadas a seguir. Cada marcação **X** indica o mês previsto de realização, sem representar uma atividade já concluída.

Figura 1 – Cronograma previsto de outubro de 2026 a junho de 2027

	Nº	Meses								
		2026			2027					
		OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Atividades	1	X	X							
	2	X	X							
	3			X						
	4				X	X				
	5						X	X		
	6							X		
	7							X	X	
	8								X	
	9	X	X	X	X	X	X	X	X	X
	10									X

X = mês previsto para realização da atividade

Fonte: Elaborado pelos autores (2026).

1. **Consolidação do pré-projeto (outubro e novembro de 2026):** uniformizar escopo, requisitos, tecnologias, modelos e conclusões parciais, além de registrar o planejamento da equipe e da avaliação.

2. **Desenvolvimento da V0 (outubro e novembro de 2026):** implementar uma prova de conceito de conexão com PostgreSQL, introspecção, preparação de contexto, geração, validação e execução de consultas. Incluir consentimento binário para amostras de até cem registros por tabela, com todas as colunas, e utilizar inicialmente um subconjunto de perguntas de teste.

3. **Revisão da V0 e planejamento da V1 (dezembro de 2026):** analisar os resultados iniciais, identificar pendências técnicas, apoiar a escolha do modelo e do provedor de linguagem e organizar o backlog e o fluxo da interface.

4. **Desenvolvimento do núcleo da V1 (janeiro e fevereiro de 2027):** integrar conexão, consentimento, contexto, geração de SQL, validação, execução com credenciais de leitura, tratamento de falhas e registro das informações de execução. O fluxo admitirá três tentativas totais, incluindo a inicial.

5. **Interface e integração desktop (março e abril de 2027):** desenvolver e integrar a interface com React, Vite e TypeScript, disponibilizando conexão, perguntas e resultados tabulares, e organizar a execução local com Electron e backend em FastAPI.

6. **Preparação da avaliação (abril de 2027):** consolidar o banco comercial fictício, as perguntas, as consultas de referência, os resultados esperados e os casos de segurança e falha. Congelar os dados e as amostras utilizados na comparação final.

7. **Estabilização e correções da V1 (abril e maio de 2027):** verificar os requisitos obrigatórios, executar testes de integração e corrigir os problemas identificados no fluxo principal e na avaliação.

8. **Avaliação comparativa e análise dos resultados (maio de 2027):** executar as perguntas com esquema e com esquema acrescido de amostras, registrar métricas e analisar respostas incorretas, tentativas de correção, custos e limitações.

9. **Documentação e acompanhamento (outubro de 2026 a junho de 2027):** atualizar o relatório, as decisões, o backlog, os registros de teste e as evidências do trabalho, com participação dos dois integrantes e maior concentração de Raissa na organização documental.

10. **Apresentação e entrega final (junho de 2027):** concluir a discussão dos resultados, revisar o relatório e a V1, preparar os materiais de apresentação e realizar a entrega para avaliação final.

A integração e a conclusão dos requisitos essenciais serão priorizadas antes da etapa final. Maio será reservado principalmente para avaliação e correções, enquanto junho será destinado à consolidação da entrega e à apresentação. As versões V2 e V3 permanecerão como possibilidades de evolução, sem integrar o compromisso obrigatório do cronograma.

5.3 Planejamento da Avaliação

A avaliação técnica será conduzida pelos autores em um banco PostgreSQL fictício do domínio comercial, contendo clientes, produtos, pedidos e itens de pedido.

Será preparado um conjunto de trinta perguntas válidas em português, distribuídas entre dez consultas simples, oito agregações, oito consultas com junções e quatro consultas que envolvam significado de negócio. Adicionalmente, serão propostas dez verificações de solicitações proibidas e cinco verificações de falhas operacionais.

Cada pergunta válida terá sua interpretação esperada, uma consulta SQL de referência e o resultado esperado documentados. As perguntas utilizadas para ajustes durante o desenvolvimento serão separadas das utilizadas na avaliação final. O banco incluirá situações que permitam identificar erros, como pedidos cancelados, valores nulos e registros próximos aos limites dos períodos consultados.

Uma resposta será considerada correta quando os dados retornados corresponderem ao resultado esperado, incluindo filtros, valores, agrupamentos e ordenação quando solicitada. A igualdade textual com o SQL de referência não será exigida. Consultas que executem sem erro, mas respondam a uma intenção diferente, serão classificadas como incorretas.

O efeito das amostras será investigado pela comparação entre duas configurações:

1. contexto contendo apenas os metadados estruturais do banco;
2. contexto contendo os mesmos metadados e amostras autorizadas de até cem registros por tabela, incluindo todas as colunas.

O modelo, as instruções, as perguntas, os dados e a política de correção serão mantidos constantes. As amostras serão fixadas para permitir a reprodução da comparação. Prevê-se realizar três execuções por pergunta em cada configuração, totalizando cento e oitenta execuções de solicitações válidas, além das verificações de segurança e falha. Cada execução respeitará o limite de três tentativas totais.

Serão registradas a taxa de respostas corretas na primeira tentativa e após as correções, a taxa de execução SQL bem-sucedida, a quantidade de tentativas, a latência e, quando disponíveis, o consumo de tokens e o custo estimado. Os resultados serão apresentados por configuração e grupo de perguntas. Os casos de segurança e falha serão contabilizados separadamente.

Como metas preliminares para discussão com o orientador, propõem-se:

- pelo menos 80% de respostas corretas às perguntas válidas na configuração com amostras, considerando até três tentativas totais;
- pelo menos 90% de solicitações válidas com execução SQL bem-sucedida nessa configuração, independentemente da correção semântica;

- nenhuma alteração do banco nas dez verificações de solicitações proibidas;
- encerramento controlado nas cinco verificações de falhas operacionais;
- ausência de envio de registros ao modelo quando o consentimento for recusado;
- realização do fluxo principal pela interface gráfica, sem utilização de terminal pelo usuário.

Essas metas deverão ser revisadas com o orientador antes da avaliação final e não constituem critérios oficiais da banca. A configuração sem amostras será avaliada separadamente, sem pressupor que a inclusão de registros necessariamente melhore o desempenho. A meta de latência e os limites de custo serão definidos após as medições da V0.

A avaliação pelos professores considerará a coerência da proposta, a demonstração do sistema, a documentação e as evidências apresentadas. Essa participação não será tratada como um estudo de usabilidade com usuários finais.

REFERÊNCIAS

DENG, N.; CHEN, Y.; ZHANG, Y. Recent advances in text-to-SQL: A survey of what we have and what we expect. In: **Proceedings of the 29th International Conference on Computational Linguistics**. Gyeongju, Republic of Korea: International Committee on Computational Linguistics, 2022. p. 2166–2187. Disponível em: <https://aclanthology.org/2022.coling-1.190/>. 29, 30

PRESSMAN, S. R. **Engenharia de Software, uma Abordagem Profissional**. [S.l.: s.n.], 2016. 35

SOMMERVILLE, I. **Engenharia de Software**. [S.l.: s.n.], 2003. 35

SOMMERVILLE, I. **Engenharia de Software, 8 edição**. [S.l.: s.n.], 2007. 35

YU, T. et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: **Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing**. Brussels, Belgium: Association for Computational Linguistics, 2018. p. 3911–3921. Disponível em: <https://aclanthology.org/D18-1425/>. 29, 30

Apêndices

APÊNDICE A – MODELO DE REQUISITOS DO SISTEMA

Este apêndice apresenta a descrição textual dos requisitos do sistema e o modelo de casos de uso da versão V1 (MVP). Estão organizados os atores envolvidos, as regras de negócio que orientam o comportamento da aplicação, os casos de uso com suas descrições e a rastreabilidade entre requisitos, casos de uso e incrementos. Os requisitos funcionais (RF) e não funcionais (RNF) referenciados estão definidos na Seção **??**.

A.1 Atores

O Quadro 1 apresenta os atores que interagem com o sistema.

Quadro 1 – Atores do sistema

Ator	**Tipo**	**Descrição**
Usuário	Primário	Pessoa que conhece o contexto dos dados, mas não necessariamente domina SQL. Configura a conexão, decide sobre o uso de amostras, formula perguntas e visualiza os resultados.
Provedor de IA	Secundário (externo)	Serviço de modelo de linguagem (por exemplo, OpenAI ou DeepSeek) que interpreta a solicitação e gera ou corrige a consulta SQL a partir do contexto recebido.
Banco PostgreSQL	Secundário (externo)	Banco de dados do usuário, acessado com as credenciais informadas, do qual são extraídos os metadados estruturais, as amostras autorizadas e os resultados das consultas.

Fonte: Elaborado pelos autores (2026).

A.2 Regras de Negócio

As regras de negócio a seguir complementam os requisitos e delimitam o comportamento esperado da aplicação.

- **RN01 – Somente leitura:** apenas consultas de leitura, compostas por uma única instrução `SELECT`, podem ser executadas.
- **RN02 – Validação independente:** a resposta do modelo de linguagem não é tratada como confiável; toda consulta gerada passa por uma camada de validação sintática e de segurança independente do modelo antes da execução.

- **RN03 – Comandos bloqueados:** comandos como `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER` e `TRUNCATE`, bem como quaisquer outros capazes de alterar registros ou estruturas, devem ser bloqueados antes da execução.
- **RN04 – Execução controlada:** as consultas aprovadas são executadas em modo somente leitura sobre o banco informado.
- **RN05 – Estrutura determinística:** os metadados estruturais (esquemas, tabelas, colunas, tipos, chaves primárias e estrangeiras) são obtidos pelo módulo de introspecção, e não por inferência do modelo de linguagem.
- **RN06 – Consentimento explícito:** nenhum registro do banco é enviado ao provedor de IA sem autorização explícita do usuário.
- **RN07 – Limite de amostragem:** quando autorizada, cada amostragem contém no máximo dez registros.
- **RN08 – Limite de correções:** uma consulta permitida que falhe pode ser reenviada ao modelo para correção em até três tentativas; esgotadas as tentativas, a operação é encerrada com informação ao usuário.
- **RN09 – Alcance definido pelas credenciais:** o sistema considera apenas as estruturas acessíveis às credenciais fornecidas pelo usuário.
- **RN10 – Rastreabilidade técnica:** latência, erros, número de tentativas e execução das consultas são registrados para apoiar a avaliação do sistema.

A.3 Casos de Uso

A Figura 2 apresenta o diagrama de casos de uso do MVP e o Quadro 2 relaciona cada caso de uso aos requisitos que ele atende.

A.3.1 UC01 – Configurar conexão com o banco

Objetivo: permitir que o usuário informe os dados de acesso a um banco PostgreSQL.

Ator: Usuário.

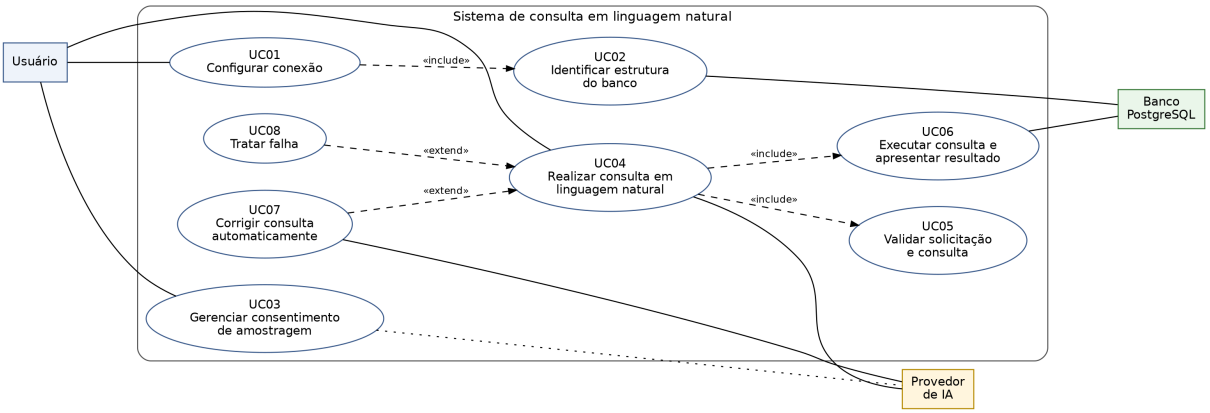
Pré-condição: a aplicação está em execução.

Pós-condição: conexão estabelecida e estrutura do banco identificada (UC02).

Fluxo principal:

1. O usuário informa os dados de conexão (servidor, porta, banco, usuário e senha).

Figura 2 – Diagrama de casos de uso do MVP (V1)



Fonte: Elaborado pelos autores (2026).

Quadro 2 – Casos de uso e requisitos atendidos

Código	Caso de uso	Requisitos	Ator(es)
UC01	Configurar conexão com o banco	RF01	Usuário
UC02	Identificar estrutura do banco	RF02, RF03	Banco PostgreSQL
UC03	Gerenciar consentimento de amostragem	RF04, RF05	Usuário, Provedor de IA
UC04	Realizar consulta em linguagem natural	RF06, RF08	Usuário, Provedor de IA
UC05	Validar solicitação e consulta	RF07, RNF01	(interno)
UC06	Executar consulta e apresentar resultado	RF09, RF10	Banco PostgreSQL
UC07	Corrigir consulta automaticamente	RF11	Provedor de IA
UC08	Tratar falha	RF12, RNF06	Usuário

Fonte: Elaborado pelos autores (2026).

2. O sistema tenta estabelecer a conexão.
3. O sistema aciona a identificação da estrutura do banco (UC02).
4. O sistema informa que o banco está pronto para receber perguntas.

Fluxo alternativo: se a conexão falhar (dados incorretos ou banco inacessível), o sistema informa o erro ao usuário e permite corrigir os dados (RNF06).

A.3.2 UC02 – Identificar estrutura do banco

Objetivo: levantar automaticamente a estrutura acessível do banco, sem depender do modelo de linguagem.

Ator: Banco PostgreSQL.

Pré-condição: conexão estabelecida (UC01).

Pós-condição: representação estruturada com esquemas, tabelas, colunas, tipos, chaves primárias e estrangeiras disponível para composição do contexto.

Fluxo principal:

1. O sistema lista os esquemas acessíveis às credenciais informadas (RF02).
2. Para cada esquema, o sistema obtém tabelas, colunas, tipos de dados, chaves primárias e chaves estrangeiras (RF03).
3. O sistema organiza as informações em uma representação estruturada.

A.3.3 UC03 – Gerenciar consentimento de amostragem

Objetivo: permitir que o usuário autorize ou recuse o envio de amostras de registros ao provedor de IA.

Ator: Usuário.

Pré-condição: estrutura do banco identificada (UC02).

Pós-condição: preferência de amostragem registrada para a sessão.

Fluxo principal:

1. O sistema informa que as amostras podem conter dados sensíveis e solicita autorização explícita (RF04).
2. O usuário autoriza a utilização das amostras.
3. Quando necessário, o sistema coleta amostras de, no máximo, dez registros por operação (RF05) e as inclui no contexto.

Fluxo alternativo: se o usuário recusar, nenhum registro é lido para composição do contexto e o sistema opera apenas com os metadados estruturais (RNF03).

A.3.4 UC04 – Realizar consulta em linguagem natural

Objetivo: obter dados do banco a partir de uma pergunta escrita em linguagem natural.

Atores: Usuário e Provedor de IA.

Pré-condição: banco conectado e estrutura identificada.

Pós-condição: resultado apresentado em formato tabular, ou falha informada de forma controlada.

Fluxo principal:

1. O usuário digita uma pergunta em linguagem natural (RF06).
2. O sistema prepara o contexto com a estrutura do banco e, se autorizadas, as amostras.
3. O sistema verifica se a solicitação é compatível com o escopo de leitura (UC05).
4. O sistema solicita ao provedor de IA a geração da consulta SQL (RF08).
5. O sistema valida a consulta gerada (UC05).
6. O sistema executa a consulta aprovada e apresenta o resultado em tabela (UC06).

Fluxos alternativos:

A1. *Solicitação incompatível:* se a pergunta exigir inserção, atualização, exclusão ou alteração estrutural, o sistema não prossegue para a execução e informa o motivo ao usuário (RF07).

A2. *Consulta reprovada na validação:* a consulta não é executada; o sistema pode solicitar nova geração dentro do limite de tentativas (UC07) ou, esgotado o limite, tratar a falha (UC08).

A3. *Erro de geração ou execução:* o sistema aciona a correção automática (UC07).

A.3.5 UC05 – Validar solicitação e consulta

Caso de uso incluído em UC04. O sistema verifica a intenção da pergunta (RF07) e, após a geração, analisa a consulta produzida por uma camada de validação independente do modelo, aprovando apenas uma instrução `SELECT` e bloqueando comandos de escrita ou de alteração estrutural (RN01 a RN03; RNF01).

A.3.6 UC06 – Executar consulta e apresentar resultado

Caso de uso incluído em UC04. O sistema executa a consulta aprovada em modo somente leitura (RF09; RN04), processa o resultado e o apresenta ao usuário em formato tabular na interface gráfica (RF10), registrando latência e demais informações de execução (RNF07).

A.3.7 UC07 – Corrigir consulta automaticamente

Caso de uso que estende UC04. Quando uma consulta permitida falha na geração ou na execução, o sistema envia o erro ao provedor de IA e obtém uma nova versão da consulta, repetindo o processo em até três tentativas (RF11; RN08). Cada tentativa é registrada para fins de avaliação (RN10).

A.3.8 UC08 – Tratar falha

Caso de uso que estende UC04. Esgotado o limite de tentativas, ou diante de falhas de conexão ou de comunicação com o provedor de IA, o sistema encerra a operação de maneira controlada e informa a falha ao usuário, sem executar consultas não aprovadas (RF12; RNF06).

A.4 Rastreabilidade dos Requisitos

O Quadro 3 relaciona os requisitos funcionais aos casos de uso e aos incrementos em que são implementados. O fluxo essencial é validado tecnicamente na V0 e integrado à interface gráfica na V1.

Quadro 3 – Rastreabilidade dos requisitos funcionais

Req.	**Descrição resumida**	**Caso de uso**	**Incremento**
RF01	Conexão com banco de dados	UC01	V0 e V1
RF02	Identificação de esquemas	UC02	V0 e V1
RF03	Introspecção estrutural	UC02	V0 e V1
RF04	Consentimento para amostragem	UC03	V1
RF05	Limitação das amostras	UC03	V1
RF06	Entrada em linguagem natural	UC04	V0 e V1
RF07	Validação da intenção	UC04, UC05	V0 e V1
RF08	Geração de consulta SQL	UC04	V0 e V1
RF09	Execução da consulta	UC06	V0 e V1
RF10	Apresentação do resultado	UC06	V0 e V1
RF11	Correção automática de consultas	UC07	V1
RF12	Tratamento de falha	UC08	V1

Fonte: Elaborado pelos autores (2026).

A.5 Critérios de Verificação dos Requisitos Não Funcionais

O Quadro 4 indica como cada requisito não funcional será verificado durante a validação do sistema.

Quadro 4 – Critérios de verificação dos requisitos não funcionais

Req.	**Requisito**	**Forma de verificação**
RNF01	Segurança	Conjunto de solicitações e consultas proibidas; medição da taxa de bloqueio correto.
RNF02	Privacidade	Inspeção do contexto enviado ao provedor, verificando a ausência de registros sem consentimento.
RNF03	Independência da amostragem	Execução do fluxo principal com a amostragem recusada.
RNF04	Modularidade	Revisão da arquitetura quanto à separação entre introspecção, contexto, geração, validação, execução e apresentação.
RNF05	Baixo acoplamento ao modelo	Substituição do provedor de IA sem alteração dos demais módulos.
RNF06	Tratamento de erros	Simulação de falhas de conexão, de provedor, de geração e de execução.
RNF07	Observabilidade	Verificação dos registros de latência, erros, tentativas e execuções.
RNF08	Usabilidade	Execução do fluxo principal pela interface desktop, sem uso de terminal ou código-fonte.

Fonte: Elaborado pelos autores (2026).

APÊNDICE B – MODELO ESTRUTURAL DO SISTEMA

Este apêndice apresenta o modelo estrutural do sistema por meio de diagramas de classes da UML. Optou-se por dois diagramas complementares: o modelo de domínio, que representa as informações manipuladas pela aplicação, e o modelo de módulos do backend, que representa as classes responsáveis por cada etapa do fluxo descrito na Seção **??**. Os nomes seguem a nomenclatura de implementação prevista e podem ser refinados durante o desenvolvimento.

B.1 Modelo de Domínio

A Figura 3 apresenta as principais classes de domínio. A estrutura do banco é obtida a partir de uma conexão e composta por esquemas, tabelas, colunas e chaves estrangeiras, obtidos de forma determinística pelo módulo de introspecção. As amostras de registros são opcionais e associadas às tabelas somente quando o usuário as autoriza. O contexto de consulta reúne a estrutura, as amostras eventuais e a pergunta, e serve de base para a geração da consulta SQL, que passa por validação e, quando aprovada, produz um resultado.

O Quadro 5 descreve a responsabilidade de cada classe.

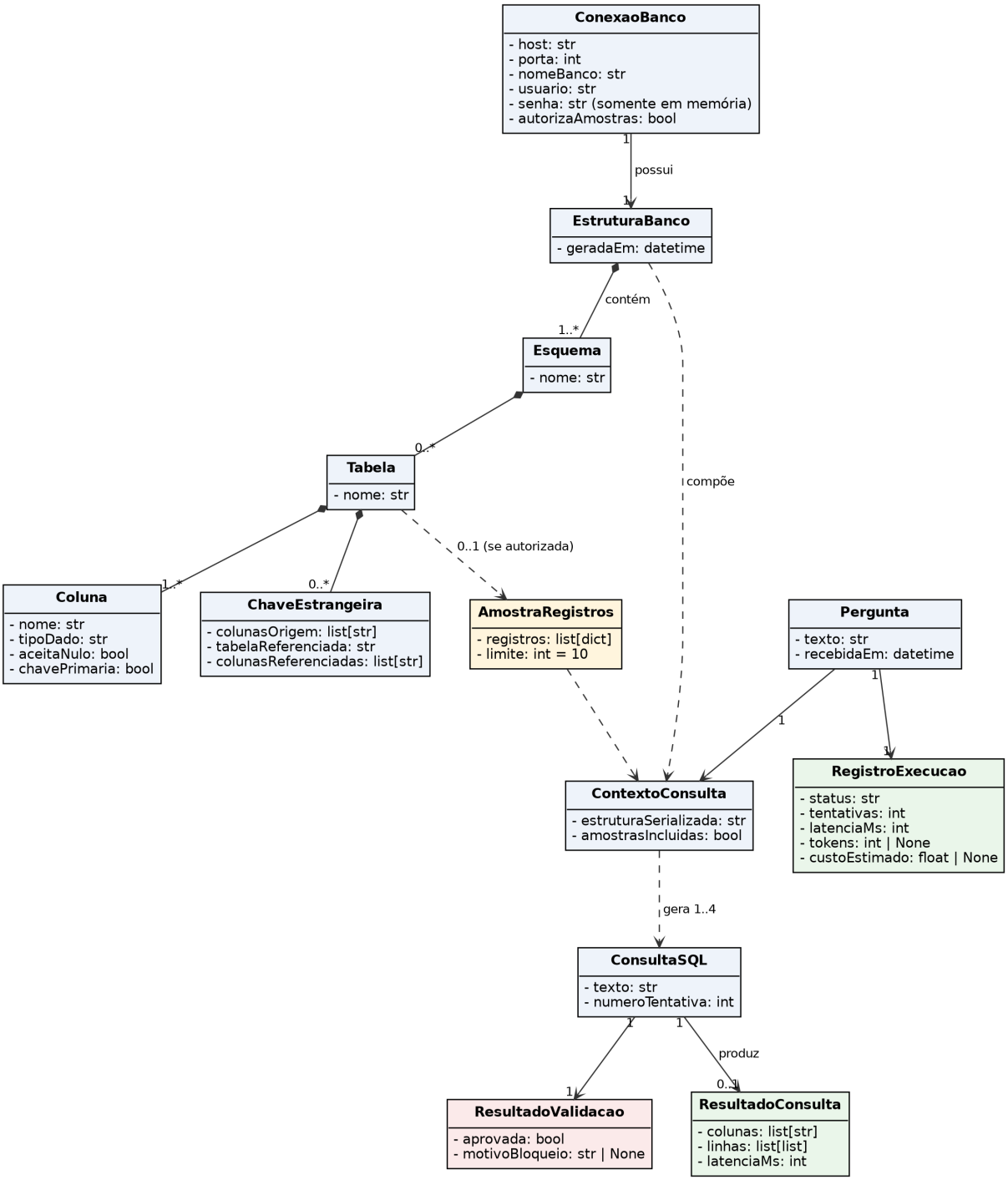
B.2 Modelo de Módulos do Backend

A Figura 4 apresenta as classes de serviço do backend em Python e FastAPI. Cada uma concentra uma etapa do fluxo, atendendo ao requisito de modularidade (RNF04). O `OrquestradorConsulta` coordena a sequência de etapas e é o único componente que conhece a ordem do fluxo. A comunicação com o provedor de inteligência artificial ocorre exclusivamente por meio da interface `ProvedorLLM`, implementada por classes específicas de cada provedor, o que permite a substituição do modelo sem alterar as demais classes (RNF05).

B.3 Considerações sobre o Modelo

As classes de domínio serão implementadas como modelos Pydantic, que definem contratos explícitos de entrada e saída da API e validam os dados trocados entre a interface e o backend. O acesso ao PostgreSQL e a introspecção utilizarão o SQLAlchemy. Diagramas complementares de comportamento, arquitetura e dados são apresentados nos apêndices seguintes.

Figura 3 – Diagrama de classes – modelo de domínio



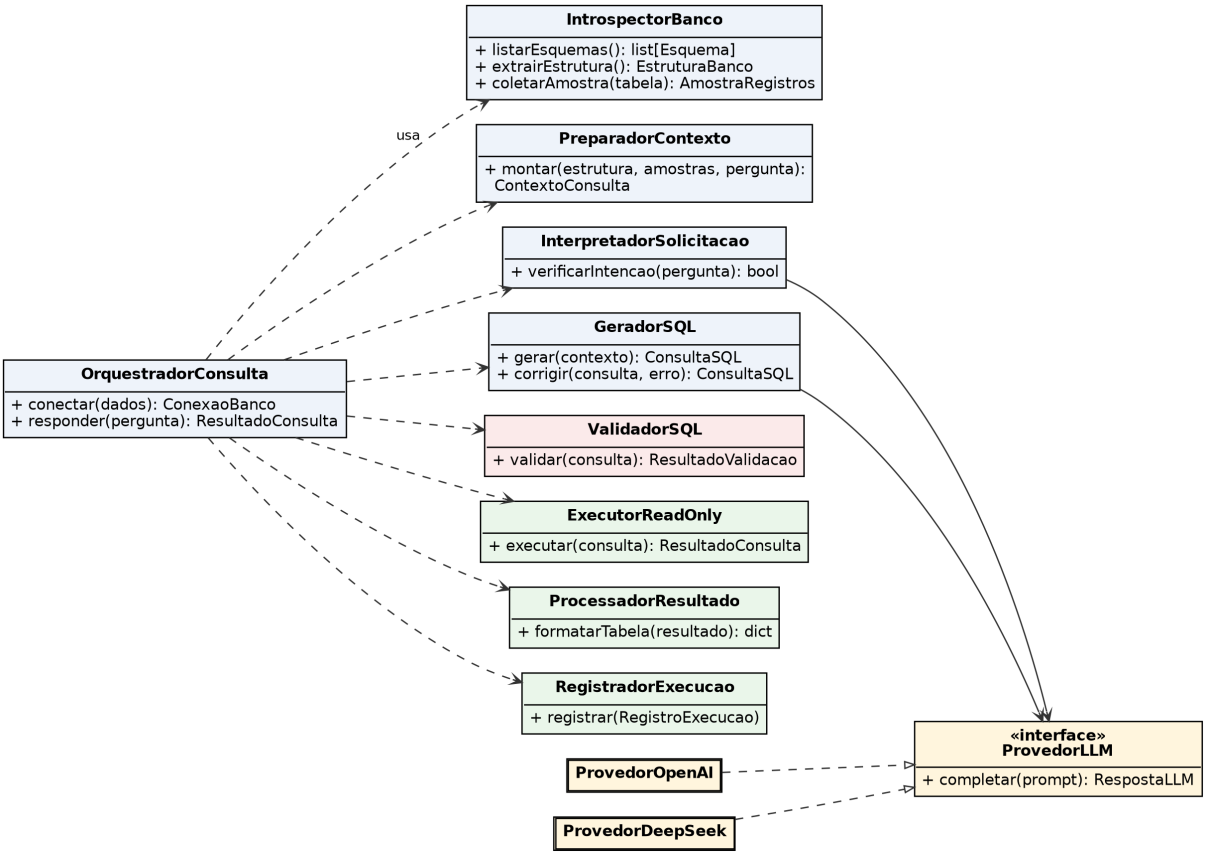
Fonte: Elaborado pelos autores (2026).

Quadro 5 – Classes do modelo de domínio

Classe	**Responsabilidade**
ConexaoBanco	Dados de acesso ao PostgreSQL informados pelo usuário e sua decisão sobre o uso de amostras (RF01, RF04). A senha é mantida apenas em memória durante a sessão.
EstruturaBanco	Representação estruturada de tudo o que foi identificado na introspecção (RF02, RF03).
Esquema, Tabela, Coluna	Elementos estruturais do banco, incluindo tipo de dado, aceitação de nulos e indicação de chave primária.
ChaveEstrangeira	Relacionamento entre tabelas, usado para que a geração de SQL reconheça junções possíveis.
AmostraRegistros	Conjunto opcional de registros, limitado a dez por amostragem (RF05), incluído no contexto somente com consentimento.
Pergunta	Solicitação em linguagem natural formulada pelo usuário (RF06).
ContextoConsulta	Contexto enviado ao provedor de IA, composto pela estrutura, pelas amostras (se houver) e pela pergunta.
ConsultaSQL	Consulta gerada pelo modelo, com indicação do número da tentativa, de modo a controlar as correções (RF11).
ResultadoValidacao	Resultado da análise de segurança: consulta aprovada ou bloqueada, com o motivo (RF07, RNF01).
ResultadoConsulta	Colunas, linhas e latência da consulta executada (RF09, RF10).
RegistroExecucao	Informações de observabilidade de cada solicitação: estado final, tentativas, latência e, quando disponíveis, tokens e custo estimado (RNF07).

Fonte: Elaborado pelos autores (2026).

Figura 4 – Diagrama de classes – módulos do backend



Fonte: Elaborado pelos autores (2026).

Quadro 6 – Classes de serviço do backend

Classe	**Responsabilidade**
OrquestradorConsulta	Coordena o fluxo completo, do recebimento da pergunta à entrega do resultado, incluindo o controle do limite de tentativas de correção.
IntrospectorBanco	Módulo de descoberta (*crawler*): lista esquemas, extrai a estrutura e, quando autorizado, coleta as amostras.
PreparadorContexto	Monta o contexto enviado ao modelo a partir da estrutura, das amostras e da pergunta.
InterpretadorSolicitacao	Identifica solicitações incompatíveis com o escopo de leitura (RF07).
GeradorSQL	Gera a consulta SQL e, em caso de erro, solicita sua correção (RF08, RF11).
ProvedorLLM (interface)	Contrato de comunicação com o modelo de linguagem, implementado por ProvedorOpenAI e ProvedorDeepSeek.
ValidadorSQL	Camada de guardrails independente do modelo: aprova apenas consultas de leitura e bloqueia as demais (RNF01).
ExecutorReadOnly	Executa a consulta aprovada em modo somente leitura (RF09).
ProcessadorResultado	Converte o resultado para o formato entregue à interface (RF10).
RegistradorExecucao	Registra latência, erros, tentativas e execuções (RNF07).

Fonte: Elaborado pelos autores (2026).

APÊNDICE C – MODELO COMPORTAMENTAL DO SISTEMA

Texto de apresentação do Apêndice - Fluxograma ou Diagrama de Sequência ou Diagrama de Estados..

APÊNDICE D – MODELO DA ARQUITETURA DO SISTEMA

Texto de apresentação do Apêndice - Diagrama de blocos de alto nível ou Diagrama de Componentes.

APÊNDICE E – MODELO DE DADOS DO SISTEMA

Texto de apresentação do Apêndice - Modelo Entidade-Relacionamento e Diagrama Entidade-Relacionamento.

APÊNDICE F – OUTRO APÊNDICE

Texto do Apêndice adicional.

Anexos

ANEXO A – TÍTULO DO ANEXO A

Texto do Anexo A.

ANEXO B – TÍTULO DO ANEXO B

Texto do Anexo B.

ANEXO C – TÍTULO DO ANEXO C

Texto do Anexo C.

ANEXO D – TÍTULO DO ANEXO D

Texto do Anexo D.

ANEXO E – TÍTULO DO ANEXO E

Texto do Anexo E.